Dynamic VPN Egress Proof — Operator Runbook (workable item #54)

Revision: 1 **Last modified:** 2026-07-01T06:42:00Z **Audience:** operator with real VPN (gluetun/WireGuard) credentials **Harness:** `tests/egress_proof/real_vpn_egress_proof.sh` **Companion:** `docs/scripts/real_vpn_egress_proof.md`

What this proves (and what is already proven)

The dynamic VPN-aware proxy has two halves:

Half	Claim	Status	Evidence
Security (fail-closed)	Tunnel down → branded `503`, **no leak**; healthd writes `DOWN`	**PROVEN** (no creds needed)	`control-plane/cmd/healthd/healthd_integration_test.go:134` (`TestIntegration_HealthdWritesDownAgainstReal` assertion `:174-176`
Functional (real egress)	With real creds, a packet routes **out through the tunnel**; egress IP via proxy **== tunnel exit** and	**CREDS-GATED** — this runbook	`tests/egress_proof/real_vpn_egress_proof.sh` — `lib/evidence.sh:assert_egress_ip:213`

Half	Claim	Status	Evidence
	!= host IP		

The security half uses a **FAKE** length-shaped WireGuard config (empty egress) on purpose, so it can prove fail-closed **without** any real key. The functional half is the only thing that needs **your real credentials** — Claude cannot and must not fabricate them (§11.4.10).

Until you run this proof with real creds, the honest project statement is: **"security proven / functional-egress creds-gated."**

Step 1 — Provide credentials (never committed, §11.4.10)

Real WireGuard keys live **only** in the gitignored `./.env` (or exported in your shell). `.env` is already in `.gitignore` (line 6) — it is never tracked. **Do not** put keys in `.env.example`, in the compose files, or anywhere git-tracked.

Copy the template and fill the **five** gluetun WireGuard variables (the exact names the `dynamic` overlay reads — `docker-compose.dynamic.yml:149-153`, `.env.example:177-182`):

```
cp .env.example .env      # if you don't already have one
chmod 600 .env            # §11.4.10 — restrict the secret file
$EDITOR .env
```

Set (values from your VPN provider's WireGuard config, e.g. Mullvad/gluetun):

```
VPN_SERVICE_PROVIDER=custom
VPN_DEFAULT_TYPE=wireguard
WIREGUARD_PRIVATE_KEY=<your real private key>
WIREGUARD_PUBLIC_KEY=<the server/peer public key>
WIREGUARD_ADDRESSES=<e.g. 10.64.0.2/32>
WIREGUARD_ENDPOINT_IP=<the tunnel endpoint IP>
WIREGUARD_ENDPOINT_PORT=<e.g. 51820>
# Optional — the known exit IP; if omitted the harness derives it
from gluetun:
EXPECTED_EXIT_IP=<the VPN exit public IP>
```

> **Note on names.** The five variables are `WIREGUARD_ENDPOINT_IP` / `WIREGUARD_ENDPOINT_PORT` (per `.env.example` + the overlay). The gluetun **container** ultimately consumes `VPN_ENDPOINT_IP` / `VPN_ENDPOINT_PORT`; the

> overlay maps the `WIREGUARD_ENDPOINT_*` `.env` names onto gluetun's env, so **set the `WIREGUARD_ENDPOINT_*` names in `.env`** and the mapping is handled for you.

Alternative — Podman secrets (rootless)

gluetun auto-reads `/run/secrets/wireguard_private_key` etc. If you prefer Podman secrets to `.env` values:

```
printf '%s' '<your real private key>' | podman secret create
        wireguard_private_key -
# ...repeat for wireguard_public_key, wireguard_addresses,
        wireguard_endpoint_ip, wireguard_endpoint_port
```

(Also create the control-plane Postgres secret if not already present: `podman secret create helixproxy_pg_password -`.)

Step 2 — Pre-flight (shared-host safety, §11.4.174)

Ensure **your** run will not collide with another owner of the data plane. The harness guards this automatically (refuses with exit 3), but check first:

```
ss -ltn | grep ':53128'
        # HTTP_PROXY_PORT must be free
podman ps --format '{{.Names}}' | grep -E 'proxy-(squid|
        gluetun)'   # must be empty
```

If a `proxy-*` stack is already up from a previous run, stop **yours** first:

```
./stop
```

The harness **never** touches operator resources `wg0-mullvad`, `lava-*`, or `:58080` (§11.4.174) — it only inspects `:53128` and the `proxy-squid`/`proxy-gluetun` names the `dynamic` overlay itself creates.

Step 3 — Run the proof

```
tests/egress_proof/real_vpn_egress_proof.sh
```

The harness will: boot `./start --dynamic` (rootless podman, §11.4.161) → wait up to `BOOT_TIMEOUT` (default 180s) for the tunnel to report a public IP → fetch the egress IP **through the proxy** and the host IP **directly** → assert **egress == tunnel exit AND egress != host** → tear the stack down (`./stop`).

Expected PASS

```
PASS: assert_egress_ip [evidence: egress=<EXIT> == exit <EXIT>, !=
host <HOST>]
PASS: issue#54 real-VPN-egress functional proof (egress != host,
== tunnel exit) [evidence: .../qa-results/issue54/
egress_via_proxy.ip]
```

Exit code `0`. Captured artefacts under `qa-results/issue54/`: `verdict.txt`, `egress_via_proxy.ip`, `host_public.ip`, `expected_exit.ip`.

Interpreting other outcomes

Outcome	Meaning	Action
`SKIP ... operator_attended`, exit 0	Creds not detected	Complete Step 1; re-run
`FAIL ... egress IP == host real IP`	Traffic **not** routed via VPN (the §15 bluff)	Check kill-switch / cred correctness
`FAIL ... egress != expected exit`	Routed, but not to the expected exit	Verify `EXPECTED_EXIT_IP` / server
`FAIL ... tunnel never came UP`	gluetun got no egress in `BOOT_TIMEOUT`	Verify keys/endpoint; raise `BOOT_TIMEOUT`
`FAIL-SAFE ... NOT booting`, exit 3	`:53128/proxy-*` contended	`./stop`, free the port, re-run

Step 4 — Record the evidence (release prep)

`qa-results/` is the gitignored raw corpus (§11.4.30). For the tracked release proof, copy the run artefacts into the curated evidence tree (§11.4.83):

```
mkdir -p docs/qa/issue54-real-egress
cp qa-results/issue54/verdict.txt docs/qa/issue54-real-egress/
# (egress_via_proxy.ip / host_public.ip / expected_exit.ip contain
        PUBLIC IPs only,
#  never key material — safe to commit as evidence; the conductor
        commits.)
```

Security note

- No secret is ever printed by the harness (§11.4.10) — presence is tested by exit status.
- `.env` and `*.ovpn` / `credentials.txt` are already gitignored; keep keys out of tracked files.
- If a key is ever suspected leaked, rotate it (§11.4.10) — do not merely delete the commit.

Sources verified

2026-07-01 — cross-checked against in-repo sources of truth: `docker-compose.dynamic.yml:127-167` (gluetun env), `.env.example:173-182` (cred var names), `start:118-124` (`--dynamic` orchestrator), `control-plane/cmd/healthd/healthd_integration_test.go:134,174` (fail-closed proof), `tests/lib/evidence.sh:213` (`assert_egress_ip`).